

دانشکده مهندسی کامپیوتر، دانشگاه صنعتی شریف

درس آزمایشگاه معماری کامپیوتر (CE۱۰۳)

گزارش کار آزمایش اول

طراحی و پیاده‌سازی جمع‌کننده دهدهی سه‌رقمی (BCD Adder)

نام و نام‌خانوادگی:

شماره دانشجویی:

استاد درس: دکتر سربازی تاریخ:

چکیده

در این آزمایش یک جمع‌کننده دهدهی سه‌رقمی (3-digit BCD Adder) در نرم‌افزار Logisim Evolution طراحی و پیاده‌سازی شده است. مدار به‌صورت سلسله‌مراتبی و از پایین به بالا ساخته شده: یک تمام‌جمع‌کننده ۱ بیتی (full_adder)، که پایه‌ی ساخت یک جمع‌کننده دهدهی سه‌رقمی دهدهی با تصحیح خودکار BCD (bcd_digit) است، و در نهایت سه نمونه از این جمع‌کننده دهدهی برای ساخت یک جمع‌کننده دهدهی سه‌رقمی کامل (main) زنجیر شده‌اند. مدار به‌طور کامل با بردارهای تست تأیید شده است.

فهرست مطالب

۱	هدف آزمایش	۳
۲	مبانی تئوری	۳
۱.۲	کدگذاری BCD	۳
۲.۲	مسئله‌ی جمع دو رقم BCD	۳
۳.۲	چرا رقم نقلی باینری کافی نیست؟	۳
۳	طراحی و پیاده‌سازی مدار	۴
۱.۳	تمام‌جمع‌کننده ۱ بیتی - full_adder	۴

۴	۱.۱.۳ پایه‌ها
۴	۲.۱.۳ منطق پیاده‌سازی
۵	۳.۱.۳ جدول درستی
۵	۲.۳ جمع‌کننده‌ی یک‌رقمی دهدهی با تصحیح bcd_digit
۵	۱.۲.۳ پایه‌ها
۷	۲.۲.۳ مثال عددی گام‌به‌گام
۷	۳.۳ مدار اصلی - main (سه رقم زنجیره‌شده)
۷	۱.۳.۳ پایه‌ها
۸	۲.۳.۳ معماری
۸	۳.۳.۳ مثال عددی

۴ صحت‌سنجی و نتایج تست ۹

۵ بحث و تحلیل طراحی ۱۰

۱۰	۱.۵ چرا از قطعه‌ی آماده‌ی Comparator استفاده نشد؟
۱۰	۲.۵ چرا ماژول جمع‌کننده‌ی ۴ بیتی جداگانه حذف شد؟
۱۰	۳.۵ هزینه‌ی سخت‌افزاری

۶ نتیجه‌گیری ۱۰

۱ هدف آزمایش

هدف از این آزمایش، آشنایی عملی با طراحی مدارهای منطقی ترکیبی سلسله‌مراتبی و پیاده‌سازی یک جمع‌کننده‌ی اعداد دهدهی کدشده به‌صورت BCD (Binary Coded Decimal) است. برخلاف جمع باینری معمولی، جمع دو رقم BCD به یک مرحله‌ی تصحیح اضافه نیاز دارد تا نتیجه در بازه‌ی مجاز () باقی بماند. اهداف مشخص این آزمایش عبارت‌اند از:

- پیاده‌سازی یک تمام‌جمع‌کننده‌ی ۱ بیتی (Full Adder) با گیت‌های پایه.
- طراحی مدار تصحیح BCD برای جمع دو رقم دهدهی به‌همراه رقم نقلی ورودی.
- ساخت یک جمع‌کننده‌ی سهرقمی با زنجیر کردن سه نمونه از جمع‌کننده‌ی یک‌رقمی.
- صحت‌سنجی مدار با بردارهای تست جامع (exhaustive test vectors).

۲ مبانی تئوری

۱.۲ کدگذاری BCD

در کدگذاری BCD، هر رقم دهدهی (۰ تا ۹) با ۴ بیت باینری نمایش داده می‌شود. برخلاف نمایش باینری خالص، در BCD کدهای ۱۰۱۰ تا ۱۱۱۱ (معادل ۱۰ تا ۱۵) بلااستفاده و نامعتبر هستند - هر رقم باید همیشه بین 0000_2 تا 1001_2 باشد.

۲.۲ مسئله‌ی جمع دو رقم BCD

اگر دو رقم BCD (هرکدام حداکثر ۹) به‌همراه یک رقم نقلی ورودی (حداکثر ۱) با یک جمع‌کننده‌ی باینری معمولی ۴ بیتی جمع زده شوند، نتیجه‌ی خام می‌تواند تا:

$$9 + 9 + 1 = 19$$

برسد که در بازه‌ی معتبر BCD (۰ تا ۹) نیست. اگر جمع خام از ۹ بیشتر باشد، باید مقدار $6 (0110_2)$ به آن اضافه شود تا:

۱. رقم باقی‌مانده در بازه‌ی صحیح BCD قرار بگیرد، و
۲. یک رقم نقلی دهدهی (نه باینری) به رقم بعدی منتقل شود.

۳.۲ چرا رقم نقلی باینری کافی نیست؟

رقم نقلی خروجی یک جمع‌کننده‌ی ۴ بیتی معمولی، تنها زمانی ۱ می‌شود که جمع خام به ۱۶ یا بیشتر برسد (سرریز باینری). اما مقادیر بین ۱۰ تا ۱۵ نیز خارج از بازه‌ی BCD هستند، در حالی که هیچ سرریز باینری‌ای رخ نداده است. بنابراین باید یک مدار تشخیص جداگانه طراحی شود که وضعیت « $9 >$ جمع خام» را به‌درستی تشخیص دهد.

۳ طراحی و پیاده‌سازی مدار

مدار در سه سطح، از پایین به بالا، طراحی شده است:

مدار	نقش	ساخته شده از
full_adder	تمام جمع‌کننده‌ی ۱ بیتی	گیت‌های OR، AND، XOR
bcd_digit	جمع‌کننده‌ی یک رقمی دهدهی با تصحیح	۸ نمونه full_adder
main	جمع‌کننده‌ی سه رقمی کامل	۳ نمونه bcd_digit

نکته‌ی مهم طراحی: در نسخه‌ی نهایی، هیچ ماژول واسط اضافه‌ای (مثل یک جمع‌کننده‌ی ۴ بیتی جدا) وجود ندارد؛ bcd_digit مستقیماً از ۸ نمونه‌ی full_adder ساخته شده تا تعداد ماژول‌ها به حداقل (۳ ماژول) برسد.

۱.۳ تمام جمع‌کننده‌ی ۱ بیتی - full_adder

۱.۱.۳ پایه‌ها

پایه	جهت	پهنا
Cin, B, A	ورودی	۱ بیت
Cout, Sum	خروجی	۱ بیت

۲.۱.۳ منطق پیاده‌سازی

مدار با ۲ گیت XOR، ۲ گیت AND و ۱ گیت OR ساخته شده است:

$X_1 = A \oplus B$	(نیمه جمع، بدون در نظر گرفتن رقم نقلی ورودی)	(۱)
$P_1 = A \wedge B$	به‌تنهایی رقم نقلی تولید می‌شود؟ A, B (آیا از جمع)	(۲)
$\text{Sum} = X_1 \oplus C_{in}$	(جمع نهایی سه ورودی)	(۳)
$P_2 = X_1 \wedge C_{in}$	رقم نقلی تولید می‌شود؟ X_1, C_{in} (آیا از جمع)	(۴)
$\text{Cout} = P_1 \vee P_2$	(رقم نقلی خروجی نهایی)	(۵)

به عبارت شهودی، Cout زمانی ۱ می‌شود که حداقل دو تا از سه ورودی A، B، Cin برابر ۱ باشند (یک رأی‌گیری اکثریت میان سه بیت) - که دقیقاً همان چیزی است که $P_1 \vee P_2$ محاسبه می‌کند.

۳.۱.۳ جدول درستی

Cout	Sum	Cin	B	A
۰	۰	۰	۰	۰
۰	۱	۱	۰	۰
۰	۱	۰	۱	۰
۱	۰	۱	۱	۰
۰	۱	۰	۰	۱
۱	۰	۱	۰	۱
۱	۰	۰	۱	۱
۱	۱	۱	۱	۱

جای خالی برای اسکرین‌شات مدار

نمای داخلی مدار full_adder در Logisim (۲ گیت XOR، ۲ گیت AND، ۱ گیت OR)

۲.۳ جمع‌کننده‌ی یک‌رقمی دهدهی با تصحیح - bcd_digit

۱.۲.۳ پایه‌ها

پایه	جهت	پهنا
B, A	ورودی	۴ بیت (رقم BCD، ۰ تا ۹)
Cin	ورودی	۱ بیت
S	خروجی	۴ بیت (رقم نتیجه، ۰ تا ۹)
Cout	خروجی	۱ بیت (رقم نقلی دهدهی)

این مدار در سه مرحله کار می‌کند که در ادامه هرکدام به‌طور کامل توضیح داده می‌شوند.

مرحله‌ی اول: جمع باینری خام. ورودی‌های A و B با یک تفکیک‌کننده (Splitter) به بیت‌های تکی $A_0..A_3$ و $B_0..B_3$ شکسته می‌شوند و در زنجیره‌ی اول از ۴ تمام‌جمع‌کننده (FA0 تا FA3) به‌صورت ripple-carry جمع زده می‌شوند:

نمونه	بیت	ورودی ها	خروجی جمع	رقم نقلی
FA0	۰ (کم ارزش)	A_0, B_0, C_{in}	S_0	CY_0
FA1	۱	A_1, B_1, CY_0	S_1	CY_1
FA2	۲	A_2, B_2, CY_1	S_2	CY_2
FA3	۳ (پر ارزش)	A_3, B_3, CY_2	S_3	C_{outH}

نتیجه‌ی این مرحله عدد ۴ بیتی $S_3S_2S_1S_0$ (جمع باینری خام، هنوز تصحیح نشده) به همراه C_{outH} (رقم نقلی خروجی این جمع ۴ بیتی، معادل وزن ۱۶) است.

مرحله‌ی دوم: تشخیص نیاز به تصحیح. باید بررسی شود که آیا عدد $S_3S_2S_1S_0$ (با احتساب C_{outH}) از ۹ بزرگتر است یا نه. چون C_{outH} به تنهایی فقط سرریز ≥ 16 را نشان می‌دهد، فرمول کمینه‌ی زیر (استاندارد در طراحی جمع‌کننده‌ی BCD) برای پوشش کامل بازه‌ی ۱۰ تا ۱۹ استفاده شده است:

$$CORR = C_{outH} \vee (S_3 \wedge S_2) \vee (S_3 \wedge S_1) \quad (۶)$$

این فرمول با تنها ۳ گیت پیاده‌سازی شده است:

سیگنال	گیت	فرمول	معنی
G1OUT	OR	$S_2 \vee S_1$	آیا یکی از بیت‌های ۱ یا ۲ روشن است؟
G2OUT	AND	$S_3 \wedge G1OUT$	آیا بیت ۳ و یکی از بیت‌های ۱/۲ همزمان روشن‌اند؟
CORR	OR	$C_{outH} \vee G2OUT$	سیگنال نهایی تصحیح

سیگنال CORR مستقیماً به پایه‌ی خروجی Cout این مدار وصل است؛ یعنی رقم نقلی دهدی، همان سیگنال تصحیح است.

چرا این فرمول درست کار می‌کند؟ اگر $C_{outH} = 1$ ، جمع خام حتماً $9 > 16 \geq$ است. اگر $C_{outH} = 0$ ، جمع خام حداکثر ۱۵ است؛ در این حالت باید $S_3 = 1$ باشد تا مقدار ≥ 8 شود. با $S_3 = 1$ اگر $S_2 = 1$ مقدار حداقل ۱۲ است (همیشه > 9)؛ اگر $S_2 = 0$ ولی $S_1 = 1$ مقدار حداقل ۱۰ است (باز هم > 9). این دقیقاً همان چیزی است که عبارت $(S_3 \wedge S_2) \vee (S_3 \wedge S_1)$ تشخیص می‌دهد.

مرحله‌ی سوم: جمع تصحیحی +6. اگر $CORR = 1$ باشد، باید مقدار ۶ (باینری ۰۱۱۰) به عدد خام اضافه شود. چون بیت‌های ۰ و ۳ عدد ۶ صفر هستند، کافی است CORR فقط به بیت‌های ۱ و ۲ اضافه شود - بدون نیاز به جمع‌کننده یا گیت AND ۴ بیتی جداگانه. این کار با زنجیره‌ی دوم از ۴ تمام‌جمع‌کننده (FAC0 تا FAC3) انجام می‌شود:

نمونه	بیت	ورودی A	ورودی B	Cin	جمع نهایی	رقم نقلی
FAC0	۰	S_0 (خام)	ZERO (۰)	ZERO	SD_0	CYC_0
FAC1	۱	S_1 (خام)	CORR	CYC_0	SD_1	CYC_1
FAC2	۲	S_2 (خام)	CORR	CYC_1	SD_2	CYC_2
FAC3	۳	S_3 (خام)	ZERO	CYC_2	SD_3 (استفاده نمی‌شود)	

بیت‌های $SD_0 \dots SD_3$ (رقم نهایی و تصحیح‌شده) با یک تفکیک‌کننده‌ی ترکیبی به پایه‌ی خروجی S وصل می‌شوند و هم‌زمان برای نمایش بصری به یک نمایشگر هفت‌قطعه‌ای (Hex Digit Display) می‌روند.

چرا رقم نقلی $FAC3$ استفاده نمی‌شود؟ چون حداکثر مقدار ممکن در این جمع، $9 + 6 = 15$ است که همیشه در ۴ بیت جا می‌شود؛ بنابراین این جمع هرگز سرریز نمی‌کند.

جای خالی برای اسکرین‌شات مدار

نمای کلی مدار bcd_digit: زنجیره‌ی اول full_adder (جمع خام)، گیت‌های تشخیص تصحیح، و زنجیره‌ی دوم full_adder (جمع +6)

۲.۲.۳ مثال عددی گام‌به‌گام

جمع $7 + 8$ (با $C_{in} = 0$) را در نظر بگیرید:

• جمع خام: $0111 + 1000 = 1111$ یعنی $S_3 S_2 S_1 S_0 = 1111$ و $C_{outH} = 0$.

• تشخیص: $G1OUT = S_2 \vee S_1 = 1 \vee 1 = 1$; $G2OUT = S_3 \wedge G1OUT = 1 \wedge 1 = 1$; $CORR = C_{outH} \vee G2OUT = 0 \vee 1 = 1$.

• تصحیح: $1111 + 0110 = 10101$ ؛ چون رقم نقلی این جمع در ۴ بیت جا نمی‌شود (نتیجه ۵ بیتی است ولی همان‌طور که گفته شد این رقم نقلی نادیده گرفته می‌شود چون از قبل توسط CORR محاسبه شده)، ۴ بیت پایین یعنی $0101 = 5$ نتیجه‌ی رقم است.

• نتیجه: $S = 5$ ، $Cout = 1$ - که دقیقاً معادل $7 + 8 = 15$ در پایه‌ی ده است (رقم ۵ با یک رقم نقلی به مرتبه‌ی بعد).

۳.۳ مدار اصلی - main (سه رقم زنجیره‌شده)

۱.۳.۳ پایه‌ها

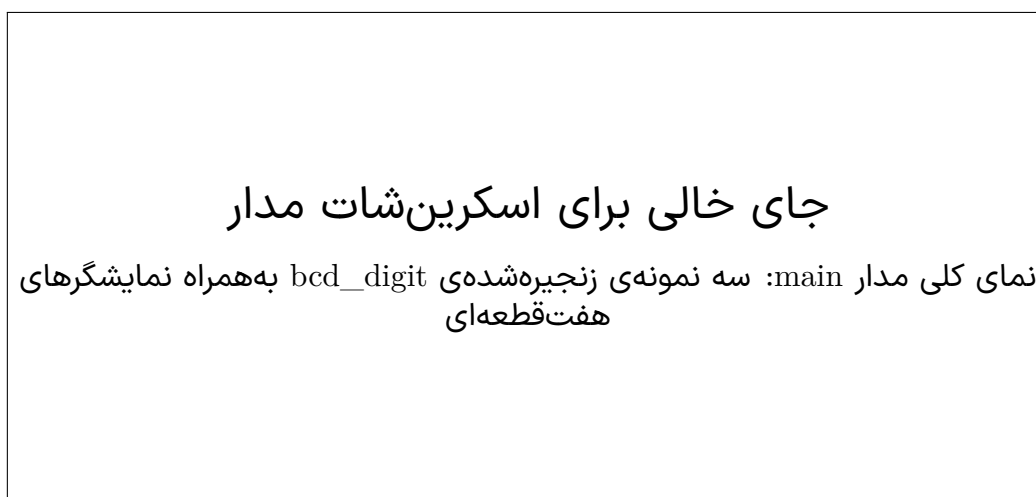
پایه	جهت	پهنا
A1, A10, A100	ورودی	۴ بیت هرکدام (رقم صدگان، دهگان، یکان عدد اول)
B1, B10, B100	ورودی	۴ بیت هرکدام (رقم صدگان، دهگان، یکان عدد دوم)
Cin	ورودی	۱ بیت
S1, S10, S100	خروجی	۴ بیت هرکدام
Cout	خروجی	۱ بیت

۲.۳.۳ معماری

سه نمونه از bcd_digit - به نام‌های digit_1، digit_10 و digit_100 - دقیقاً مانند جمع دستی چند رقمی روی کاغذ، به صورت زنجیره‌ای به هم وصل شده‌اند:

نمونه	رقم	A	B	Cin	Cout می‌رود به
digit_1	یکان	A1	B1	Cin (ورودی مدار)	تونل Cout1
digit_10	دهگان	A10	B10	تونل Cout1	تونل Cout10
digit_100	صدگان	A100	B100	تونل Cout10	پایه Cout مدار

هر رقم روی یک نمایشگر Hex Digit Display جداگانه نشان داده می‌شود و Cout نهایی به یک LED وصل است (در صورت سرریز از عدد ۹۹۹، روشن می‌شود).



۳.۳.۳ مثال عددی

$$123 + 456 = 579$$

با ورودی‌های $A100=1, A10=2, A1=3$ و $B100=4, B10=5, B1=6$ ، مدار باید خروجی $S100=5, S10=7, S1=9$ و $Cout=0$ تولید کند.

جای خالی برای اسکرین‌شات مدار

اجرای مدار در Logisim با ورودی‌های 123 و 456 و مشاهده‌ی خروجی 579 روی نمایشگرها

۴ صحت‌سنجی و نتایج تست

هر سه مدار با بردار تست (.vec) و از طریق خط فرمان Logisim در حالت headless (-test-vector) تست شدند:

نتیجه	تعداد حالت تست‌شده	مدار
✓ ۸ / ۸	۸ (تمام حالت‌های ممکن 2^3)	full_adder
✓ ۲۰۰ / ۲۰۰	۲۰۰ (تمام ترکیب‌های معتبر $A \times B \times C_{in}$)	bcd_digit
✓ ۶ / ۶	۶ (نمونه‌های smoke test)	main

عدد ۲۰۰ برای bcd_digit از این‌جا می‌آید: $A \in \{0..9\}$ (۱۰ حالت) $B \in \{0..9\}$ (۱۰ حالت) $C_{in} \in \{0, 1\}$ (۲ حالت) $200 = \text{حالت} - \text{یعنی تمام ورودی‌های معتبر BCD پوشش داده شده‌اند، نه فقط چند نمونه.}$

جای خالی برای اسکرین‌شات مدار

خروجی خط فرمان تست بردار برای مدار bcd_digit: Passed: 200, Failed: 0

۵ بحث و تحلیل طراحی

۱.۵ چرا از قطعه‌ی آماده‌ی Comparator استفاده نشد؟

در طراحی‌های اولیه، برای تشخیص «آیا جمع خام از ۹ بزرگ‌تر است»، از یک قطعه‌ی آماده‌ی Comparator در حالت unsigned استفاده می‌شد. اما این قطعه از نظر سخت‌افزاری بیش از حد لازم است: با فرمول کمینه‌ی معادل (۶)، همان تشخیص با تنها ۲ گیت (AND و OR) به جای یک مقایسه‌گر کامل ۴ بیتی قابل انجام است. این جایگزینی هم تعداد قطعات را کم می‌کند و هم مدار را ساده‌تر و قابل‌فهم‌تر می‌کند.

۲.۵ چرا ماژول جمع‌کننده‌ی ۴ بیتی جداگانه حذف شد؟

در نسخه‌ی قبلی طراحی، یک مدار واسط به نام adder4 (جمع‌کننده‌ی ۴ بیتی عمومی، ساخته شده از ۴ full_adder) وجود داشت که bcd_digit از دو نمونه آن استفاده می‌کرد. در نسخه‌ی نهایی، این لایه‌ی واسط حذف و ۸ نمونه‌ی full_adder مستقیماً داخل bcd_digit قرار گرفتند. نتیجه: دقیقاً ۳ ماژول در کل پروژه (main, bcd_digit, full_adder)، بدون هیچ لایه‌ی سلسله‌مراتبی اضافه.

۳.۵ هزینه‌ی سخت‌افزاری

هر رقم bcd_digit از ۸ تمام‌جمع‌کننده (۴۰ گیت پایه) به علاوه ۳ گیت تصحیح تشکیل شده است؛ در مجموع، مدار سهرقمی main از ۲۴ تمام‌جمع‌کننده و ۹ گیت تصحیح ساخته شده - یک مبادله‌ی معمول میان سادگی طراحی سلسله‌مراتبی و تعداد قطعات.

۶ نتیجه‌گیری

در این آزمایش، یک جمع‌کننده‌ی دهدهی سهرقمی به طور کامل و سلسله‌مراتبی طراحی و پیاده‌سازی شد. نکات کلیدی به دست آمده:

- جمع دو رقم BCD نیاز به یک مرحله‌ی تصحیح دارد چون جمع باینری خام می‌تواند تا ۱۹ برسد در حالی که هر رقم BCD حداکثر ۹ است.
- رقم نقلی باینری معمولی برای تشخیص این وضعیت کافی نیست؛ باید بازه‌ی ۱۰ تا ۱۵ نیز جداگانه تشخیص داده شود.
- با یک فرمول بولی کمینه (فرمول (۶))، این تشخیص تنها با ۳ گیت - بدون نیاز به مقایسه‌گر کامل - قابل پیاده‌سازی است.
- طراحی سلسله‌مراتبی (main → bcd_digit → full_adder) امکان استفاده‌ی مجدد و تست مستقل هر سطح را فراهم می‌کند.
- مدار نهایی با ۲۰۸ حالت تست (۸ برای full_adder و ۲۰۰ برای bcd_digit) به طور کامل و بدون خطا تأیید شده است.